

Lecture 10 Gaussian Discriminant Analysis, Unsupervised Learning, and K-Means

Announcements

- Naïve Bayes exercise, along with the solution, has been posted on Canvas
- HW 2 Due today
- HW 3 will be released soon and Due 10/19
- Focusing on Naïve Bayes and GDA

Last Lecture

- Generative model for text classification
- Bag-of-Words Representation
- Naïve Bayes principle
- Model probability of seeing words in each label class $P_{\theta}(x_j = 1 | y = k)$
 ψ_{jk}
- $P_{\theta}(x|y = k)$: product of the occurrence probabilities of each words in x in class k

$$P_{\theta}(x|y = k) = \prod_{j=1}^d P_{\theta}(x_j | y = k)$$

- Prior belief about seeing the label class

Learn ϕ from data: $P_{\theta}(y = k) = \phi_K$

Last Lecture

- MLE solution for Naïve Bayes can be solved in closed form:
- The loss function decomposes into terms that only involve ψ_{jk} and ϕ_K

$$\begin{aligned}\ell &= \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}) \\ &= \underbrace{\sum_{k=1}^K \sum_{j=1}^d \sum_{i: y^{(i)}=k} \log P(x_j^{(i)} | y^{(i)}; \psi_{jk})}_{\text{all the terms that involve } \psi_{jk}} + \underbrace{\sum_{i=1}^n \log P(y^{(i)}; \phi)}_{\text{all the terms that involve } \phi} .\end{aligned}$$

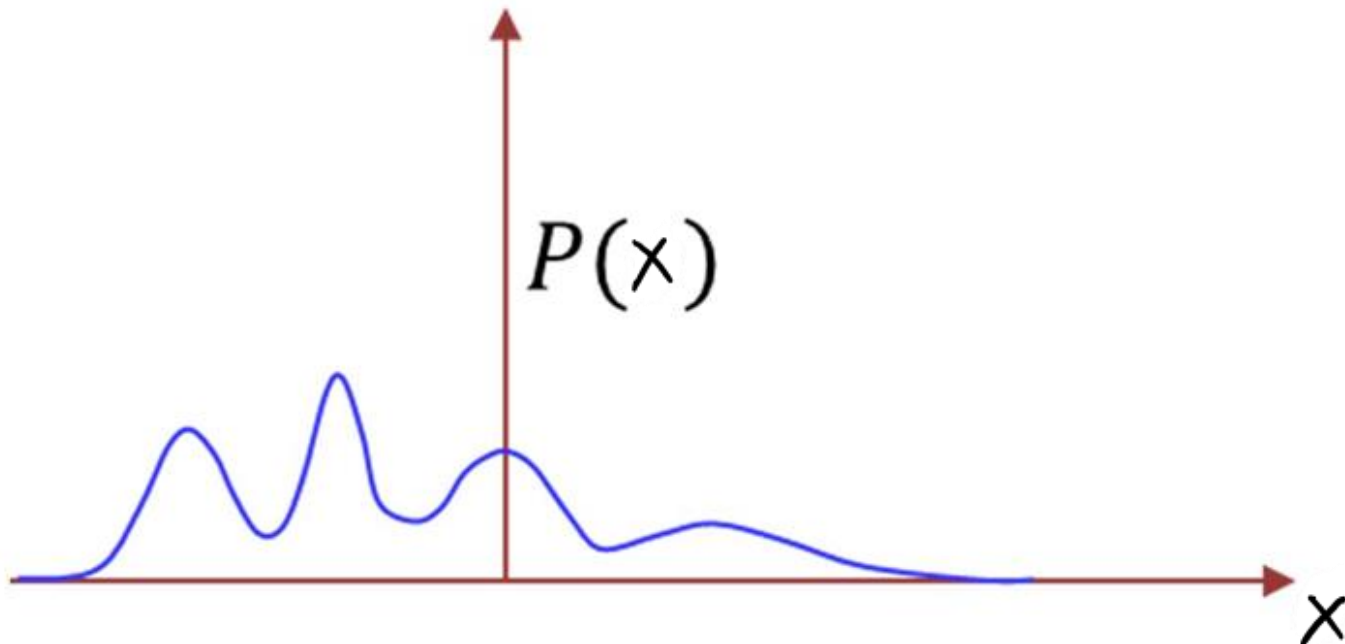
Last Lecture

- MLE solution for Naïve Bayes can be solved in closed form:
- The loss function decomposes into terms that only involve ψ_{jk} and ϕ_K
- The maximum likelihood of Bernoulli distribution is the empirical count
- ψ_{jk} : the proportion of documents in class k contains word j
- The optimal ϕ_K is the proportion of data points with class k (n_k) in the training set

Gaussian Discriminant Analysis

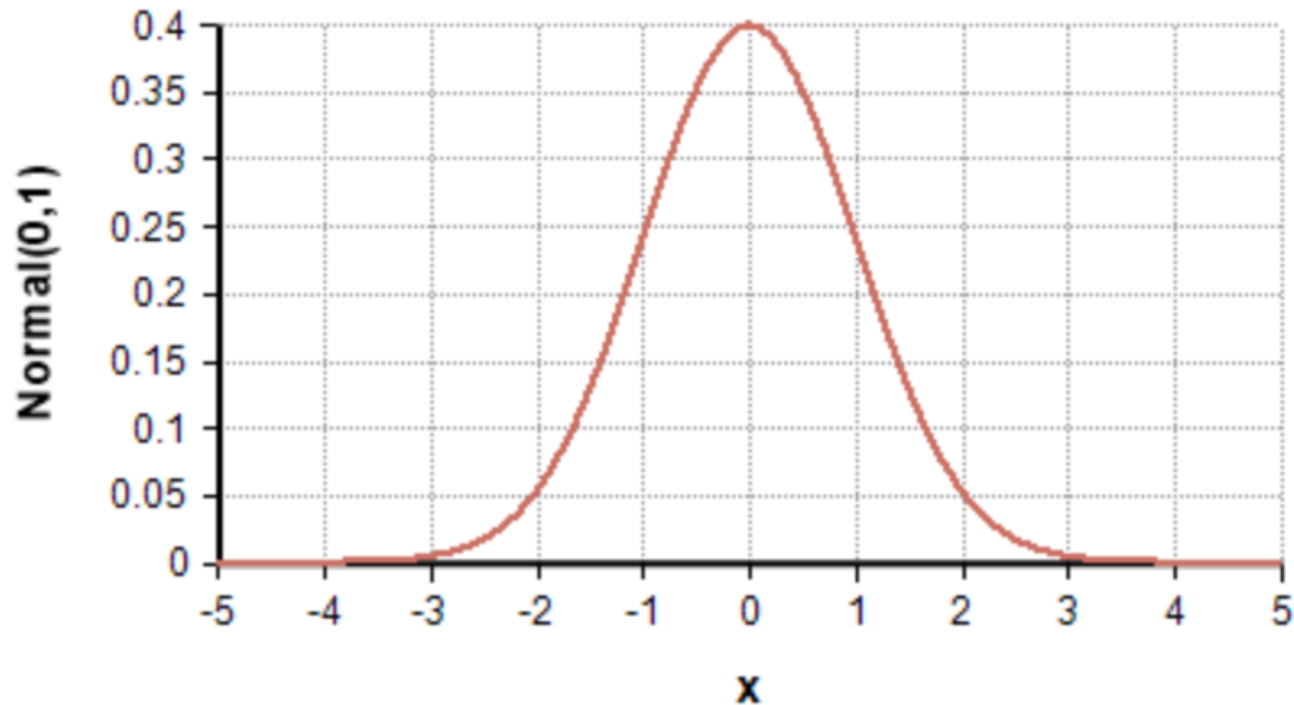
Gaussian Mixture Models (GMMs)

- Bernoulli Naïve Bayes is a particular way of modeling $P(\mathbf{x}|y)$ for text classification
- If the features are continuous, we want to use a better way to model continuous random variables
- For example, the distribution of X is of the following shape



Gaussian Mixture Models

- Standard Gaussian Distribution $N(u, \sigma^2)$



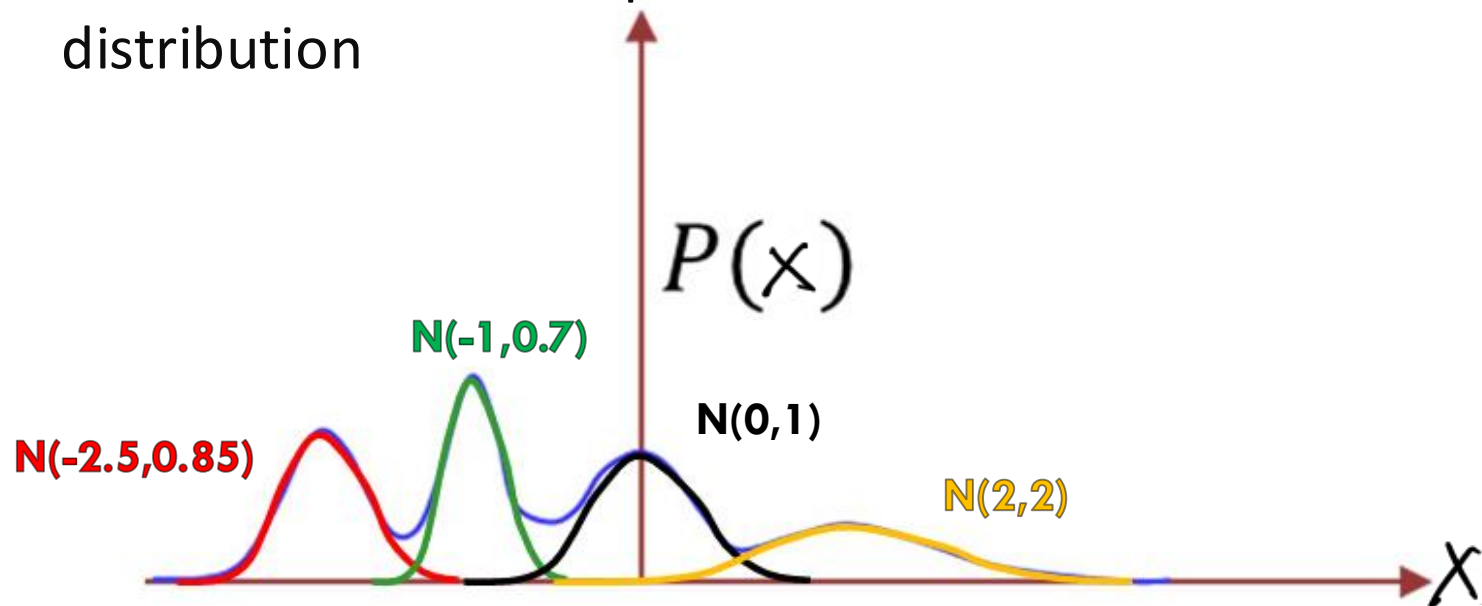
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Gaussian Mixture Models (GMMs)

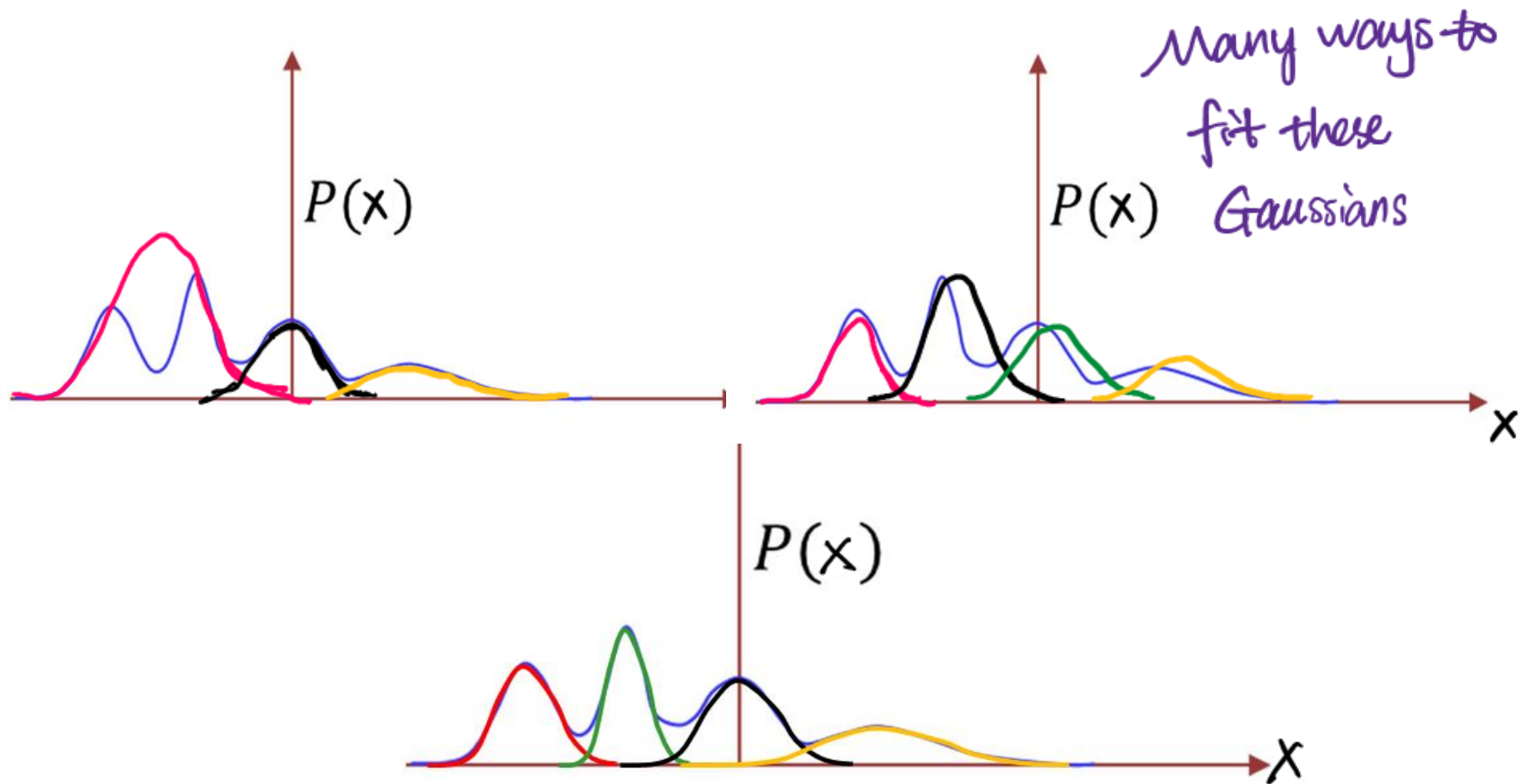
- We model it as a mixture of Gaussians (GMMs)
- Weighted sum of Gaussians

$$P(x) = \sum_k P(k) N(x; \mu_k, \sigma_k^2)$$

- The weights $p(k)$ sum to 1, each representing the likelihood that a data point is from each Gaussian distribution



Gaussian Mixture Models (GMMs)



Goal: given the samples from the original distribution, how to determine the parameters of GMM to best fit them?

GMM as Generative Classification Model

Mixture of Gaussians (GMMs): With $p(k)$'s sum to 1

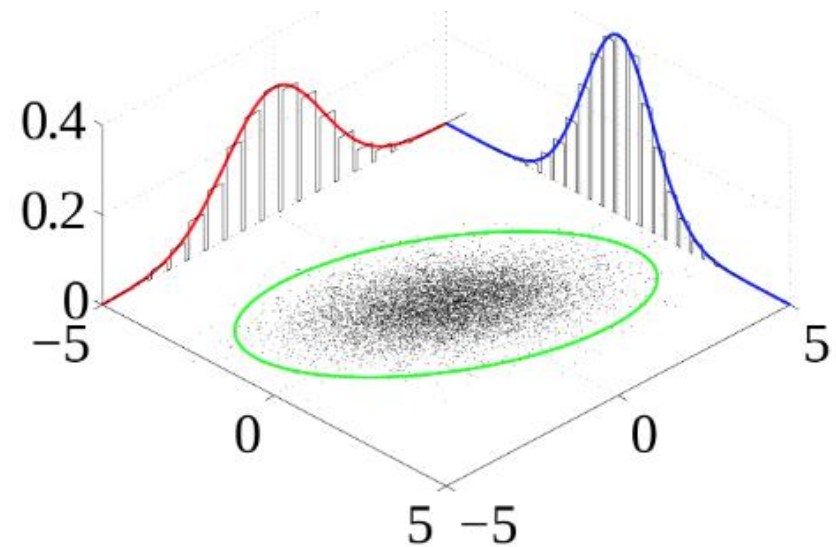
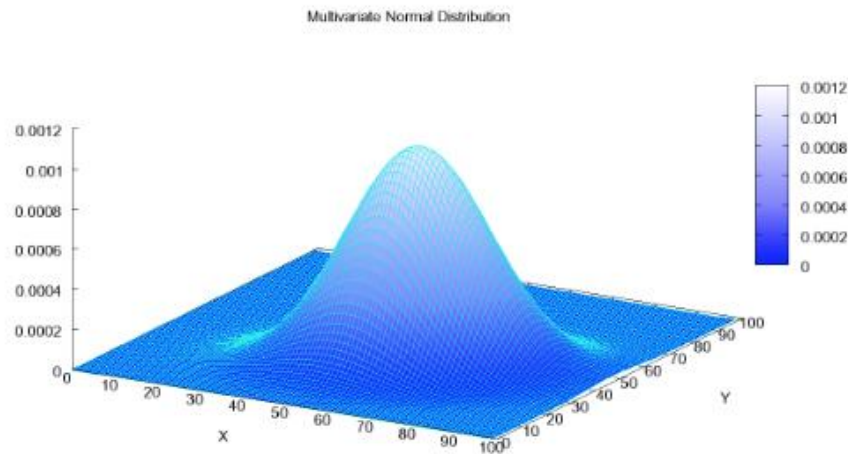
$$P(x) = \sum_k P(k)N(x; \mu_k, \sigma_k^2)$$

- The number of mixtures corresponds to the number of label classes:
- $P(k) \rightarrow P(Y = k)$
- Each mixture corresponds to the conditional distribution of feature given label
- $N(x; \mu_k, \sigma_k^2) \rightarrow P(x|Y = k)$

$$P(x) = \sum_k P(Y = k)P(x|Y = k)$$

Multivariate Gaussian Distribution $N(\mathbf{u}, \Sigma)$

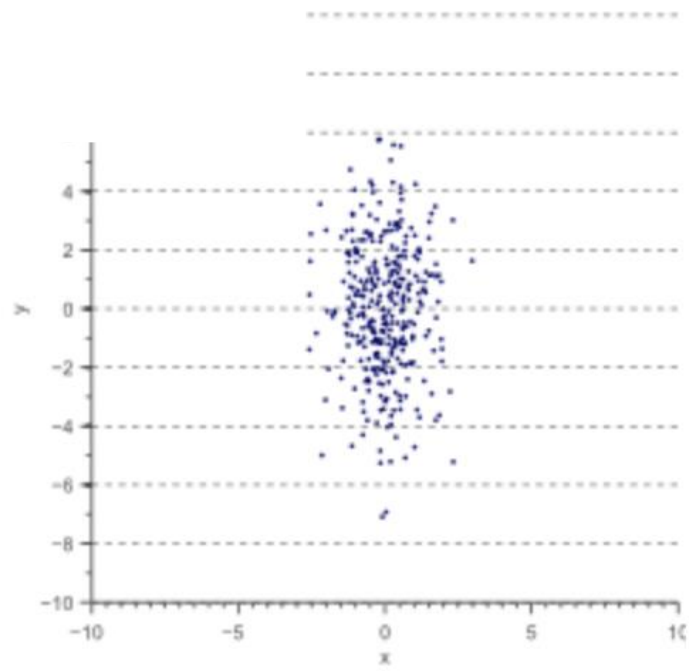
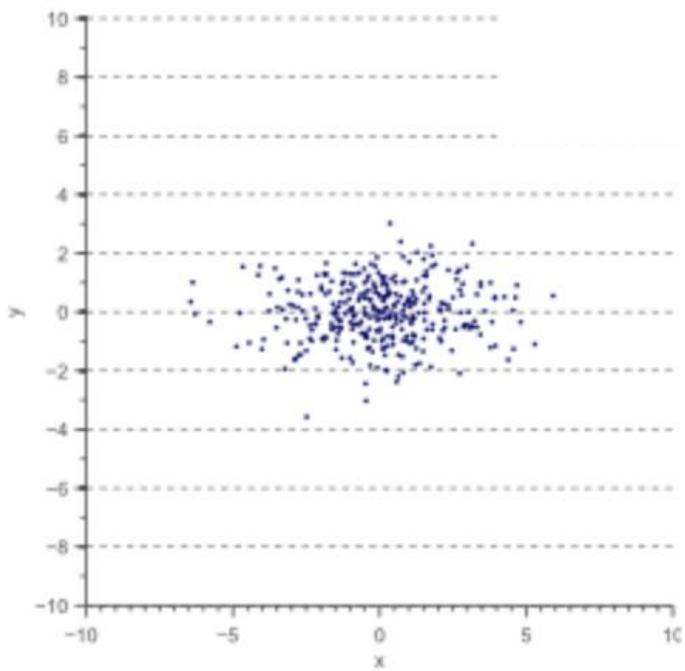
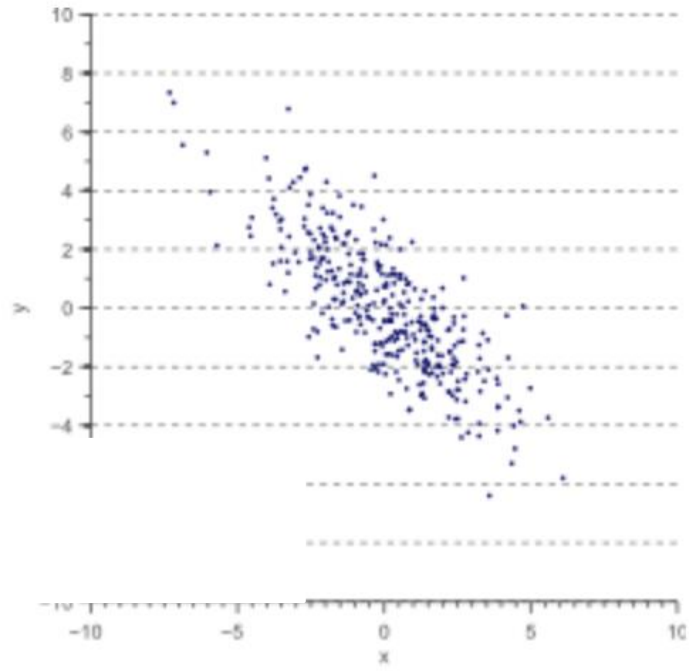
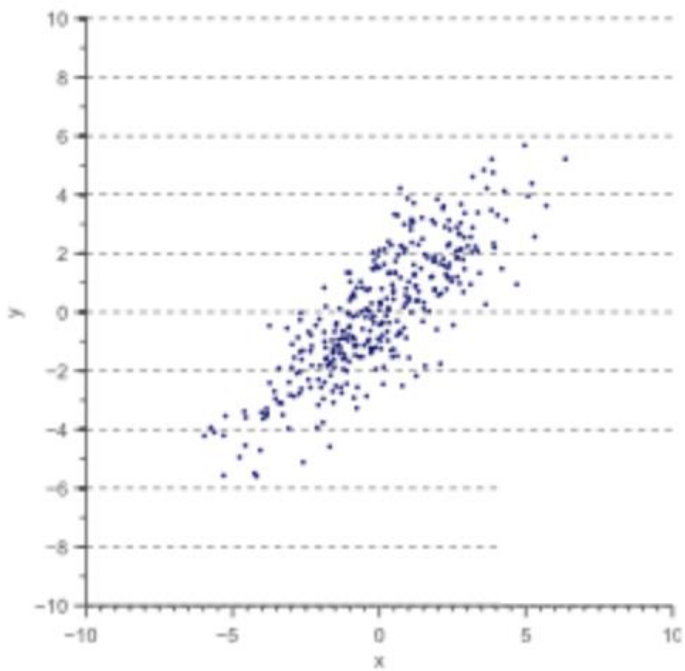
- Multivariate Gaussian Distribution $N(\mathbf{u}, \Sigma)$
- $\mathbf{u}$: vector of length d
- Σ : d by d matrix denote the covariance



Covariance Matrix

$$\text{cov}(X, Y) = \text{E} [(X - \text{E}[X])(Y - \text{E}[Y])]$$

$$\begin{array}{cc} & \begin{array}{c} x \\ y \end{array} \\ \begin{array}{c} x \\ y \end{array} & \left[\begin{array}{cc} \text{var}(x) & \text{cov}(x, y) \\ \text{cov}(x, y) & \text{var}(y) \end{array} \right] \end{array}$$



Gaussian Discriminant Analysis (GDA)

Mixture of Gaussians (GMMs): With $p(k)$'s sum to 1

$$P(\mathbf{x}) = \sum_k P(Y = k)P(\mathbf{x}|Y = k)$$

Where $P(\mathbf{x}|Y = k): N(\mathbf{x}; \mu_k, \Sigma_k)$

- Gaussian discriminant analysis: fit the parameters μ_k, Σ_k to be empirical means and variances of each label class k .

GDA through maximum likelihood estimation

- Log-likelihood function

$$\ell(\theta) = \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)})$$

$$= \sum_{i=1}^n \log P_{\theta}(x^{(i)} | y^{(i)}) + \sum_{i=1}^n \log P_{\theta}(y^{(i)})$$

$$= \underbrace{\sum_{k=1}^K \sum_{i: y^{(i)}=k} \log P(x^{(i)} | y^{(i)}; \mu_k, \Sigma_k)}_{\text{all the terms that involve } \mu_k, \Sigma_k} + \underbrace{\sum_{i=1}^n \log P(y^{(i)}; \vec{\phi})}_{\text{all the terms that involve } \vec{\phi}} .$$

GDA through maximum likelihood estimation

- Solving for $\vec{\phi} = (\phi_1, \dots, \phi_K)$ for $k=1, 2, \dots, K$:

$$\max_{\vec{\phi}} \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}) = \max_{\vec{\phi}} \sum_{i=1}^n \log P_{\theta}(y^{(i)}; \vec{\phi})$$

Can be carried out independently for each label class

- The optimal ϕ_K is the proportion of data points with class k (n_k) in the training set

$$\frac{\phi_k}{\sum_l \phi_l} = \frac{n_k}{n}$$

GDA through maximum likelihood estimation

- Each μ_k, Σ_k for $k=1,2,\dots,K$ is found in:

$$\begin{aligned} \max_{\mu_k, \Sigma_k} \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}) \\ &= \max_{\mu_k, \Sigma_k} \sum_{l=1}^K \sum_{i: y^{(i)}=l} \log P_{\theta}(x^{(i)} | y^{(i)}; \mu_l, \Sigma_l) \\ &= \max_{\mu_k, \Sigma_k} \sum_{i: y^{(i)}=k} \log P_{\theta}(x^{(i)} | y^{(i)}; \mu_k, \Sigma_k). \end{aligned}$$

- Carry out optimization over μ_k, Σ_k independently of all the other parameters

Learning μ_k, Σ_k through MLE

- Optimizing for each μ_k, Σ_k for $k=1,2,\dots,K$:

$$\begin{aligned} \max_{\mu_k, \Sigma_k} \sum_{i:y^{(i)}=k} \log P_{\theta}(x^{(i)} | y^{(i)}; \mu_k, \Sigma_k) \\ = \max_{\mu_k, \Sigma_k} \sum_{i:y^{(i)}=k} \log \mathcal{N}(x^{(i)} | \mu_k, \Sigma_k) \end{aligned}$$

- Calculating the maximum likelihood by
 - Plugging in the expression for the multivariate Gaussian distribution
 - Compute the derivative and setting it to zero

Learning μ_k, Σ_k through MLE

- Optimizing for each μ_k, Σ_k for $k=1,2,\dots,K$:

$$\begin{aligned} \max_{\mu_k, \Sigma_k} \sum_{i:y^{(i)}=k} \log P_{\theta}(x^{(i)} | y^{(i)}; \mu_k, \Sigma_k) \\ = \max_{\mu_k, \Sigma_k} \sum_{i:y^{(i)}=k} \log \mathcal{N}(x^{(i)} | \mu_k, \Sigma_k) \end{aligned}$$

- The MLE of μ_k, Σ_k are just the empirical means and covariances of each class

$$\begin{aligned} \mu_k &= \frac{\sum_{i:y^{(i)}=k} x^{(i)}}{n_k} \\ \Sigma_k &= \frac{\sum_{i:y^{(i)}=k} (x^{(i)} - \mu_k)(x^{(i)} - \mu_k)^T}{n_k} \end{aligned}$$

Are the following statements on GMM correct?

In classification, GMM assumes that the conditional distribution of the feature given the label is Gaussian.

0

The number of mixtures corresponds to the number of label classes, and we can find the parameter of the best fit through MLE, which can be solved in closed form.

0

When making a prediction $P(y|x)$ given the model trained by GDA, we apply Bayes rule by selecting the class label that maximizes $P_{\theta}(x|y) P_{\theta}(y)$.

0

GDA is equivalent of naive Bayes but for continuous data.

0

Unsupervised Learning

- A dataset without labels
 - Learn interesting structures within the data
- Clusters of related datapoints
 - E.g., customer segmentation.

Customer Segmentation

- Segmentation:
 - Subdivision of customers into a relatively small number of groups that share similar characteristics
 - How many groups is enough to capture customer behaviors?
 - Features: customers' needs, past behaviors, and demographic profiles
- Use case: targeted marketing
- Algorithm: clustering

Unsupervised Learning

- A dataset without labels
 - Learn interesting structures within the data
- Clusters of related datapoints
 - E.g., customer segmentation.
- Outliers, i.e., particularly unusual or interesting data points.

Outlier Detection

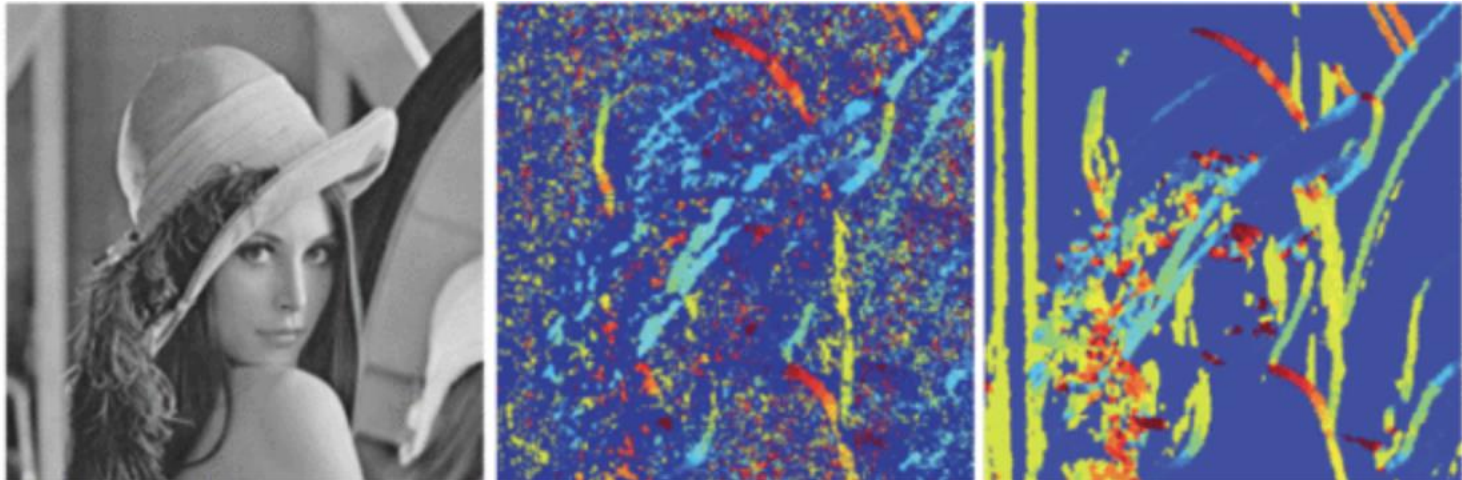
- Given a new data point
 - Want to know whether this new point is normal or not
- Features:
 - A summary of traffic logs on a computer network
 - The state of a machine in a factory
- Model the data generating distribution
- Density estimation

Unsupervised Learning

- A dataset without labels
 - Learn interesting structures within the data
- Clusters of related datapoints
 - E.g., customer segmentation.
- Outliers, i.e., particularly unusual or interesting datapoints.
- Denoised signals.

Denoised signals

- Image with added Gaussian noise



- Assume data generated from different Gaussian distributions
 - Use Gaussian Mixture Models to cluster data
 - Remove the cluster that is labeled as noise

Unsupervised Learning

- A dataset without labels
 - Learn interesting structures within the data
- Clusters of related datapoints
 - E.g., customer segmentation.
- Outliers, i.e., particularly unusual or interesting datapoints.
- Denoised signals.
- Dimensionality reduction

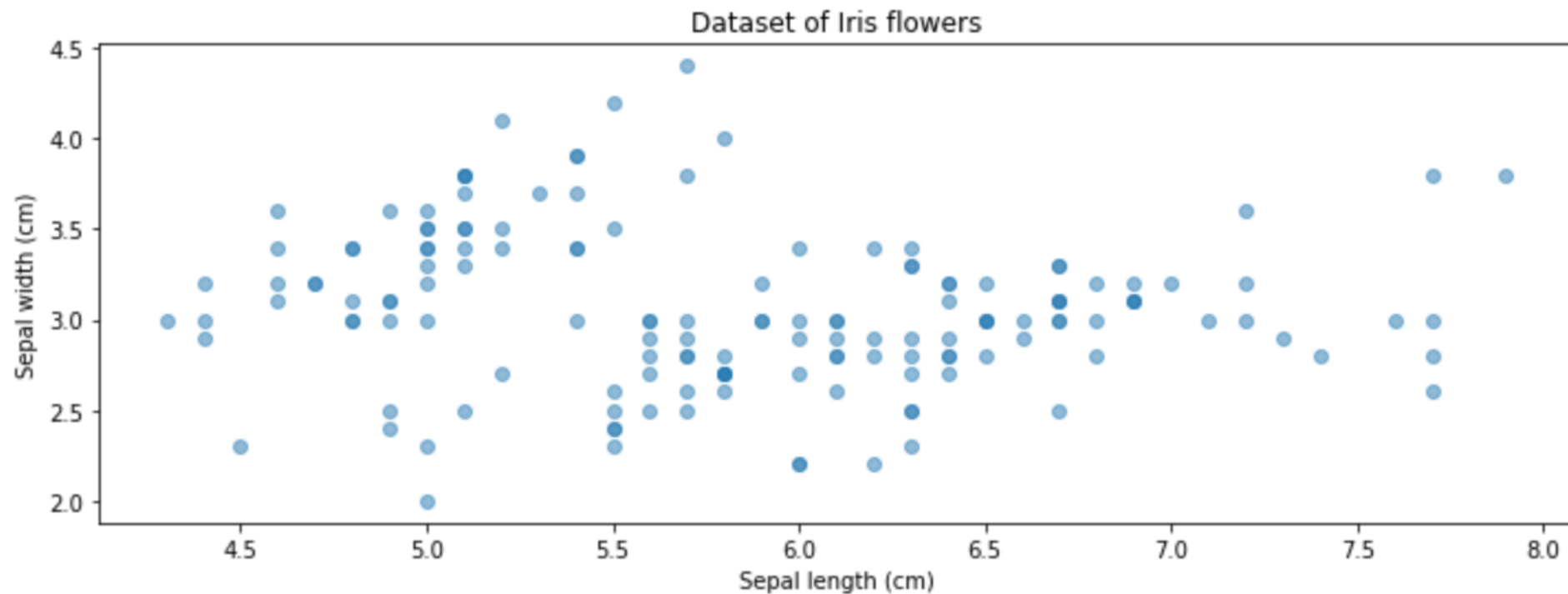
Data and Visualization

2D Visualization (not using labels)

```
from matplotlib import pyplot as plt
plt.rcParams['figure.figsize'] = [12, 4]

# Visualize the Iris flower dataset
plt.scatter(iris.data[:,0], iris.data[:,1], alpha=0.5)
plt.ylabel("Sepal width (cm)")
plt.xlabel("Sepal length (cm)")
plt.title("Dataset of Iris flowers")
```

```
Text(0.5, 1.0, 'Dataset of Iris flowers')
```



K-Means

K-Means

- Find K hidden clusters in the data.

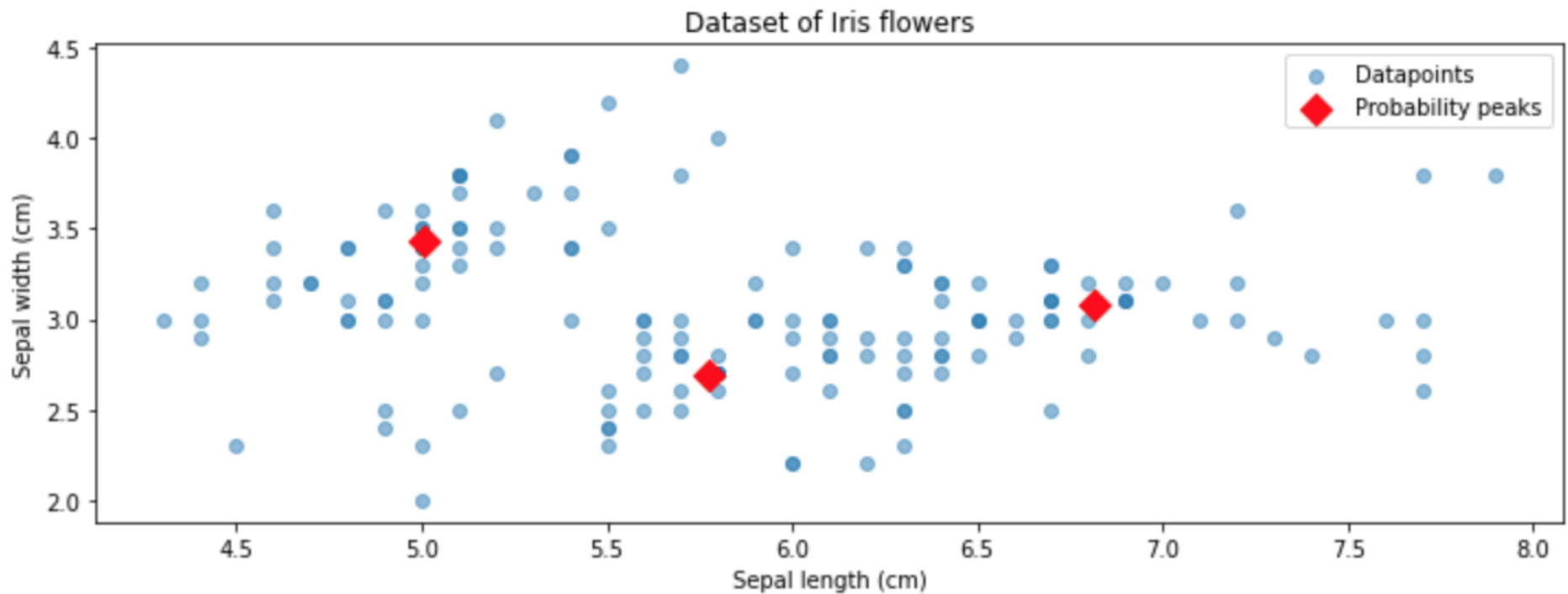
```
# fit K-Means with K=3  
from sklearn import cluster  
model = cluster.KMeans(n_clusters=3)  
model.fit(iris.data[:, [0, 1]])
```

KMeans(n_clusters=3)

```
# display the clusters in 2D  
plt.scatter(iris.data[:, 0], iris.data[:, 1], alpha=0.5)  
plt.scatter(model.cluster_centers_[:, 0],  
            model.cluster_centers_[:, 1], marker='D', c='r', s=100)  
plt.ylabel("Sepal width (cm)")  
plt.xlabel("Sepal length (cm)")  
plt.title("Dataset of Iris flowers")  
plt.legend(['Datapoints', 'Probability peaks'])
```

K-Means

- Obtain three clusters



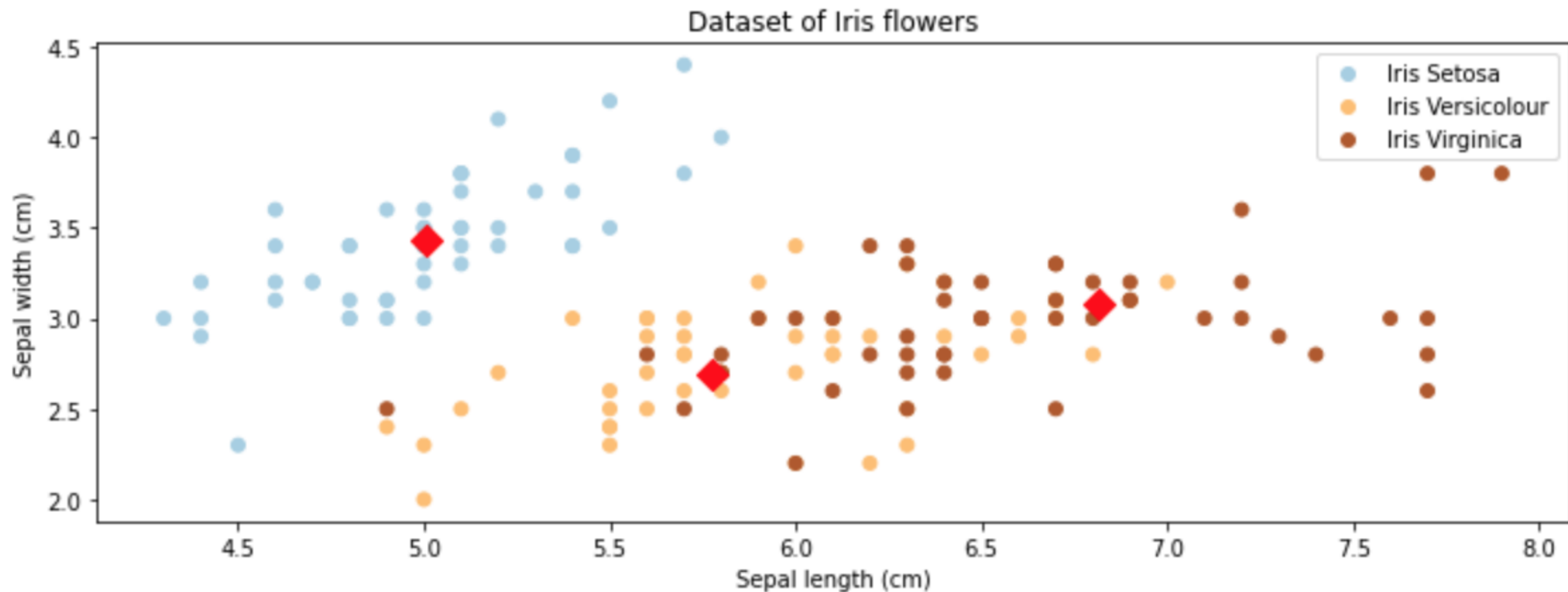
K-Means

- Plot labels to visually examine how good the clustering algorithm is

```
p1 = plt.scatter(iris.data[:,0], iris.data[:,1],  
                 alpha=1, c=iris.target, cmap='Paired')  
plt.scatter(model.cluster_centers_[0],  
            model.cluster_centers_[1], marker='D', c='r', s=100)  
plt.ylabel("Sepal width (cm)")  
plt.xlabel("Sepal length (cm)")  
plt.title("Dataset of Iris flowers")  
plt.legend(handles=p1.legend_elements()[0],  
           labels=['Iris Setosa', 'Iris Versicolour', 'Iris Virginica'])
```

K-Means

- Plot labels to visually examine how good the centers returned by the clustering algorithm are



Unsupervised Learning Notations

- We define of size n a dataset for unsupervised learning as

$$\mathcal{D} = \{x^{(i)} \mid i = 1, 2, \dots, n\}$$

- Each $x^{(i)} \in \mathbb{R}^d$ denotes an input, a vector of d attributes or features.

Components of Unsupervised Learning Algorithms

- A **model class**
 - The set of possible unsupervised models we consider.
- An **objective** function
 - Defines how good a model is
 - Cannot use/won't have labels
- An **optimizer**
 - Finds the best predictive model in the model class according to the objective function

k-Means Clustering *Principles*

K-Means Model

- We can think of the model returned by K -Means as a function

$$f : \mathcal{X} \rightarrow \mathcal{Z}$$

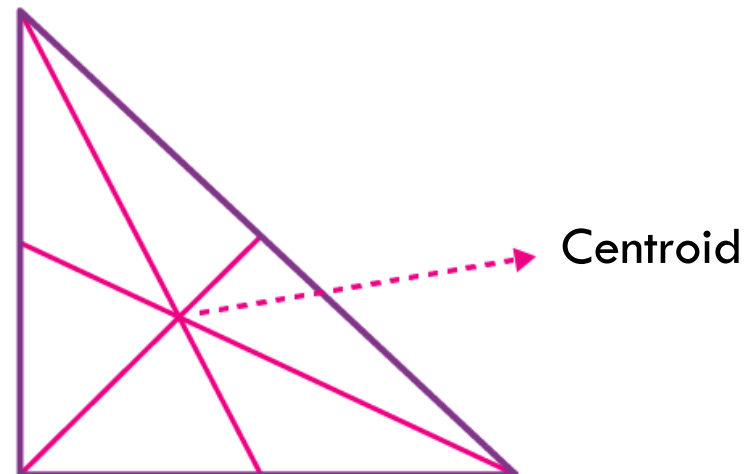
- f assigns each input x to a cluster ID $z \in Z = \{1, 2, \dots, K\}$.

K-Means Objective

- Find centroids c_k such that
- the distance between the points and their closest centroid is minimized:

$$J(f) = \frac{1}{n} \sum_{i=1}^n ||x^{(i)} - \text{centroid}(f(x^{(i)}))||,$$

- where $\text{centroid}(k)=c_k$ denotes the centroid for cluster k .



k-Means Clustering

Set number
of clusters

- **Select k clusters**

Initialize

- **Randomly select k centroid locations**
 - A “centroid” can be thought of as a point that is “representative” of each cluster
 - A centroid is not necessarily a data point

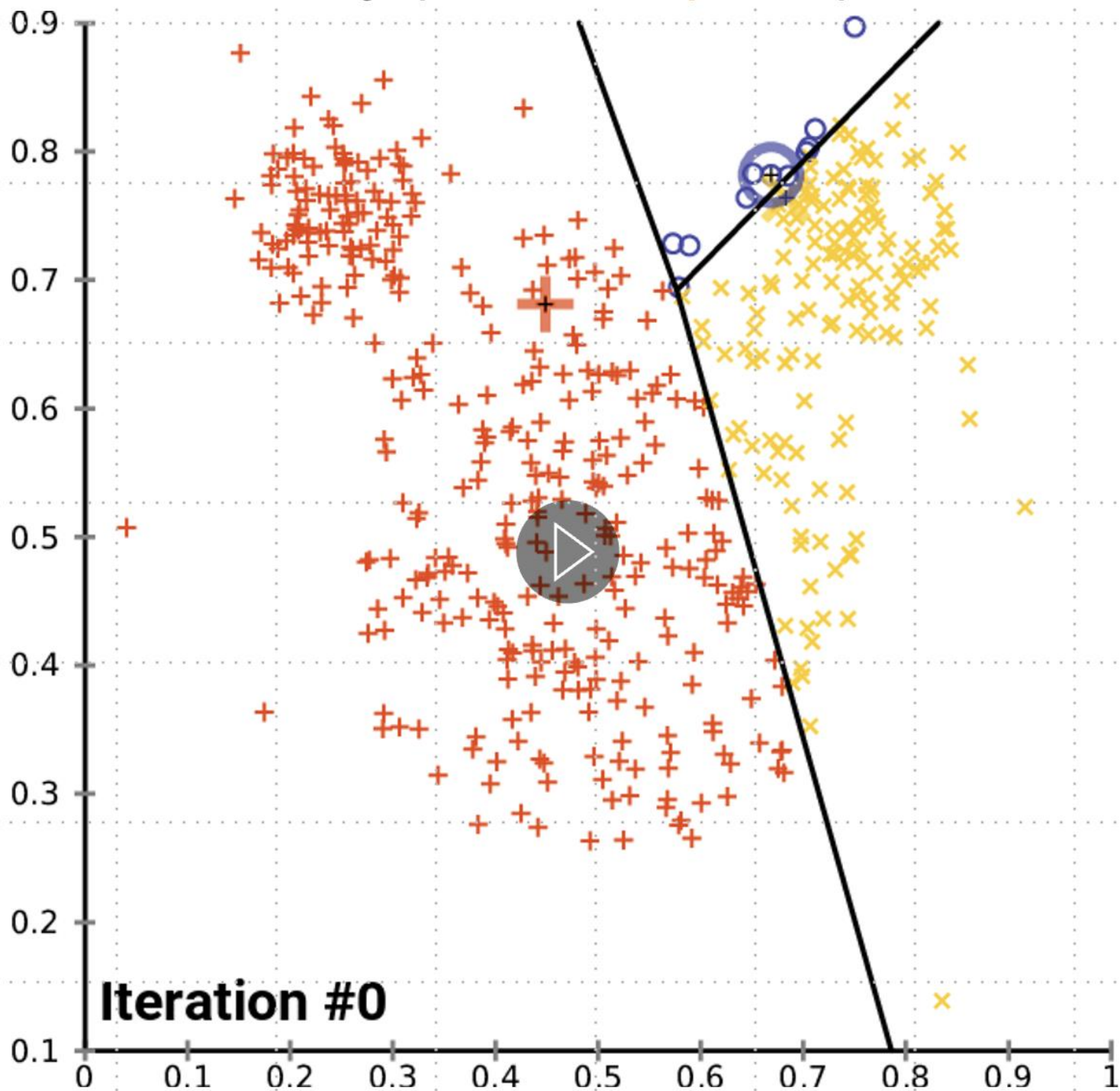
Iterate

- **Repeat until convergence**
 - Assign each observation to the nearest centroid
 - Recalculate centroids as average of assigned observations

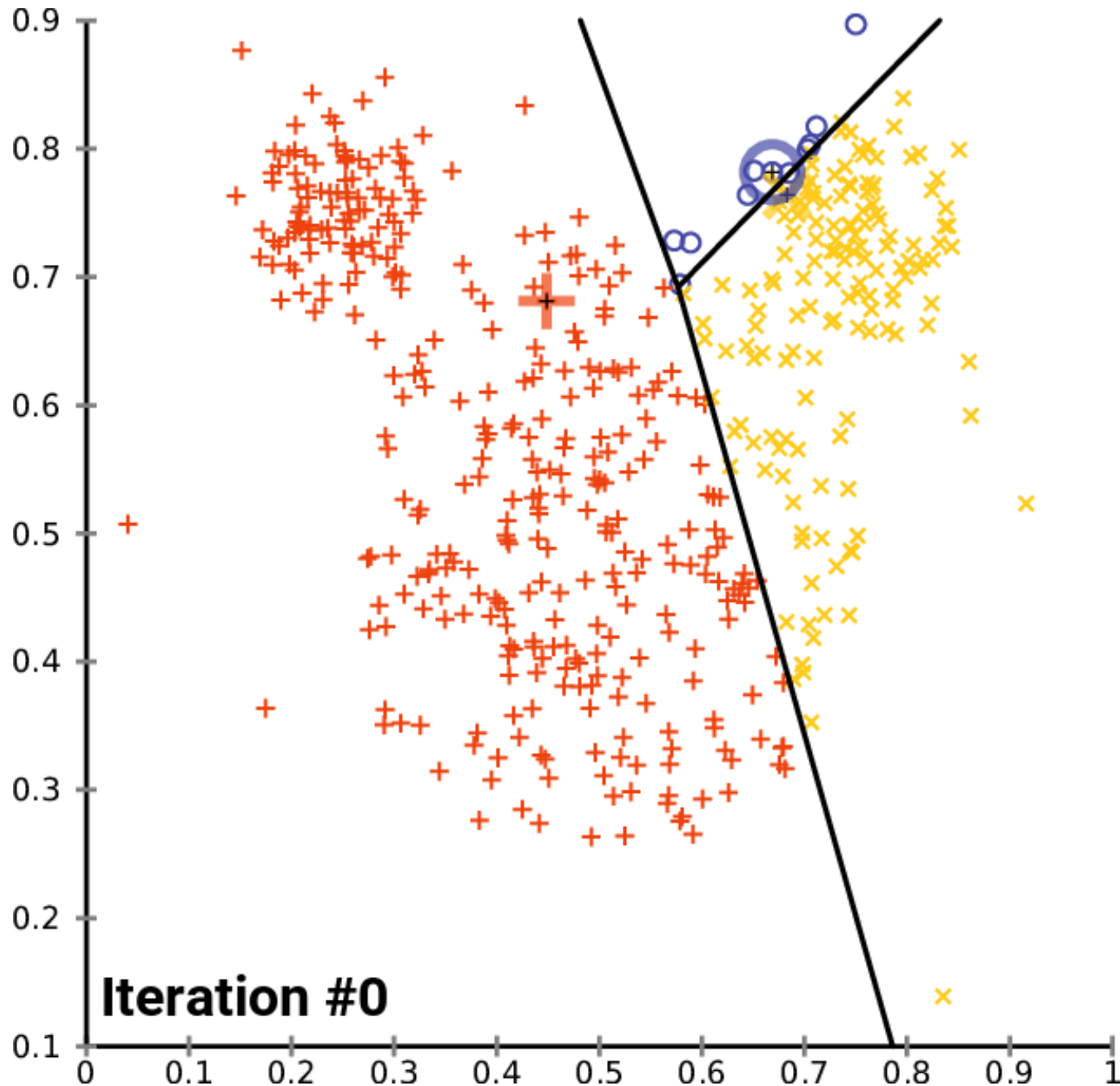
Terminate

- **Terminate when no observations get reassigned**

III

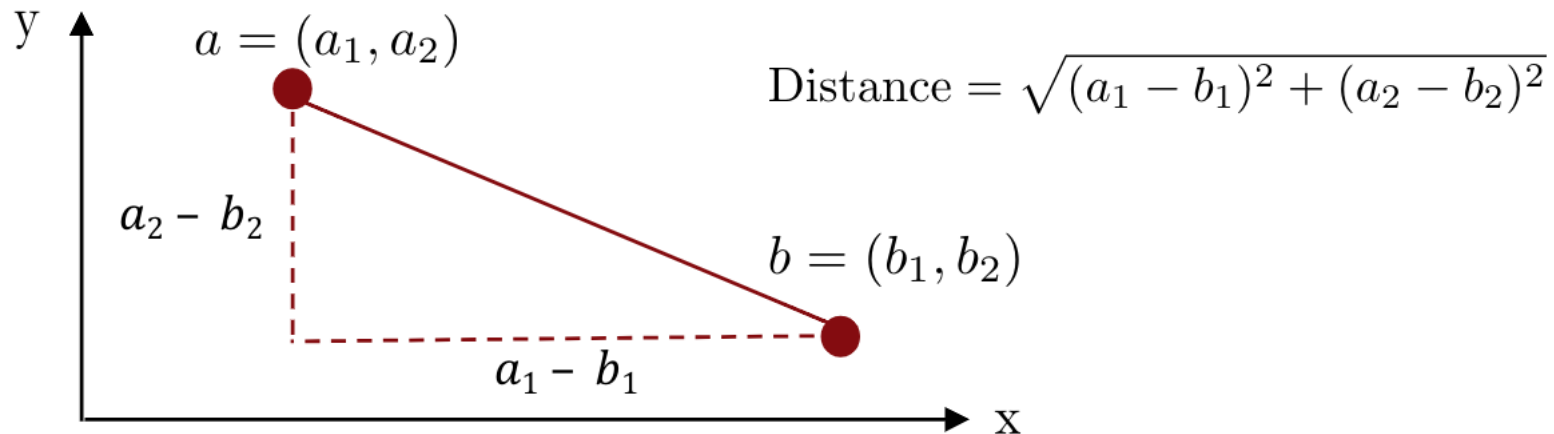


Illustrated visually (from [Wikipedia](#)):



Measuring Distance between 2 Points

- Clustering relies on the distance between pairs of observations
- Typically, Euclidean distance, L2 norm



- Can be directly generalized to instances with more variables

$$\text{Distance} = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_6 - b_6)^2}$$

Are the following questions on K-means correct?



The performance of K-means is sensitive to the choice of units of the variables. For example, measuring time in terms of days vs years

0

The L2 distance metric is primarily driven by the variables of a smaller scale, impacting the performance of K-means

0

It is hard to compare different units in L2 distance, e.g., comparing days and miles.

0

Normalization in K-Means

- Normalized variables are expressed in terms of “number of standard deviations from mean”

$$\text{Normalized data} = \frac{\text{Data} - \text{Sample mean}}{\text{Sample standard deviation}}$$

k-Means Clustering

Set number
of clusters

- Select k number of clusters

Initialize

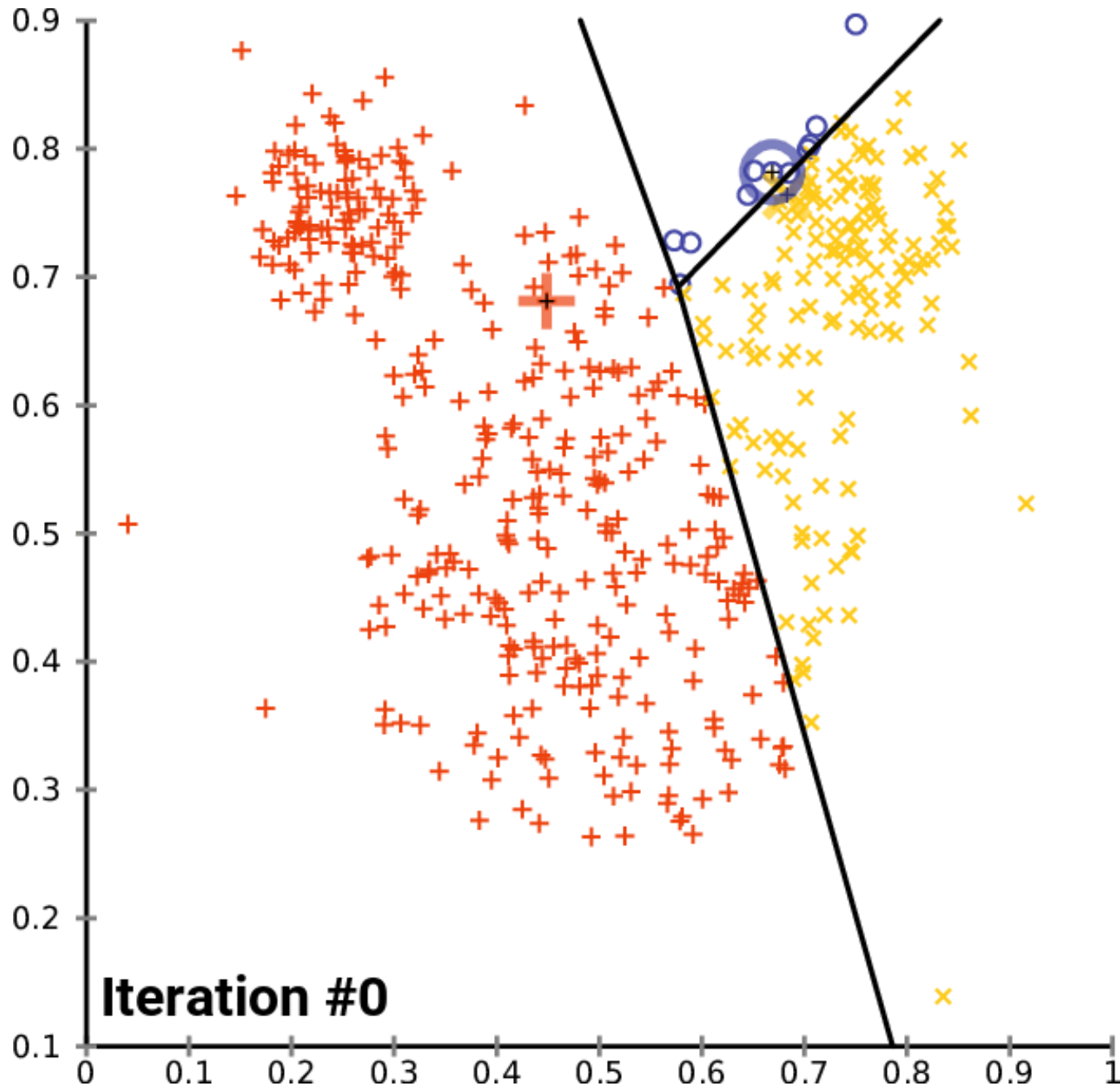
- Randomly select k centroid locations

Iterate

- Repeat until convergence
- Update $f(x)$ such that
$$f(x(i)) = \operatorname{argmin}_k ||x(i) - c_k||_2$$
- is the cluster of the closest centroid to $x(i)$.
- Set each c_k to be the center (the average vector) of the its cluster $\{x(i) | f(x(i))=k\}$
- Terminate when no observations get reassigned

Terminate

Illustrated visually (from [Wikipedia](#)):



Are the following statements on K-means and clustering correct?



K-means will return the exact same solution every time it is run on the same dataset with different initial centroids.

0

K-means performs the best when the variables are correlated (i.e., they form ellipsoid-shaped clusters) and have different variances across clusters

0

k-means clustering can get stuck at local minima

0

Measuring clustering quality is hard and relies on heuristics

0

The clustering assignment returned by k-means has a probabilistic interpretation that represents the confidence of the assignment

0